

Week 5 Home Work

Submission Deadline: 22 May 2026, 11:59 PM

1. Shopping Cart Manager

Problem Statement:

- Take item names as input one by one (3 items).
- Store all items in a list using `append()`.
- Print the full cart and the total number of items.

Acceptance Criteria:

- Start with an empty list.
- Use `append()` to add each item.
- Print the list.
- Print the count using `len()`.

Expected Output:

```
Enter item 1: milk
Enter item 2: eggs
Enter item 3: bread
Your cart: ['milk', 'eggs', 'bread']
Total items: 3
```

2. Hospital Waiting Queue

Problem Statement:

- Start with a pre-defined patient queue: ['Rin', 'Sam', 'Yuki'].
- Add a new patient 'Leo' to the back of the queue.
- Call the first patient (remove from front) and print their name.
- Print the remaining queue and its length.

Acceptance Criteria:

- Use `append()` to add Leo.
- Use `pop(0)` to call and remove the first patient.
- Store the removed name and print it in a sentence.
- Print updated list and use `len()` for count.

Expected Output:

```
Now calling: Rin
Remaining queue: ['Sam', 'Yuki', 'Leo']
Patients waiting: 3
```

3. Cinema Seat Booking

Problem Statement:

- Given seats = [10, 11, 12, 13, 14, 15, 16].
- Print the first and last seat using indexing (no len()).
- Slice 3 seats starting at index 2 and store as booked_seats.
- If booked_seats has exactly 3 seats, print confirmation; otherwise print 'Booking failed'.

Acceptance Criteria:

- Use seats[0] for first seat and seats[-1] for last.
- Use seats[2:5] to extract exactly 3 seats.
- Use len(booked_seats) == 3 inside an if/else.
- Print both the seat numbers and the confirmation message.

Expected Output:

```
First seat: 10 | Last seat: 16
Booked seats: [12, 13, 14]
Booking confirmed for 3 seats!
```

4. Student Name Formatter

Problem Statement:

- Take 3 student names as input (may be in any case).
- Store each name in a list after converting to title case.
- Print the final list and the total student count.

Acceptance Criteria:

- Use append() to add each name.
- Apply .title() before storing.
- Print the list and use len() for count.

Expected Output:

```
Enter name 1: ANANYA SHARMA
Enter name 2: kenji tanaka
Enter name 3: OMAR FAROUK
Students: ['Ananya Sharma', 'Kenji Tanaka', 'Omar Farouk']
Total students: 3
```

5. News Feed — Latest 3 Headlines

Problem Statement:

- Given a list of headlines stored oldest-first.
- Slice the last 3 headlines into a variable called recent.
- Print the total headline count and recent count.
- Print each recent headline numbered and in title case.
- If total headlines are fewer than 3, print 'Not enough news yet'.

Acceptance Criteria:

- Use a negative slice `headlines[-3:]` for recent.
- Use `len()` for both counts.
- Access each item by index and apply `.title()`.
- Use `if/else` on `len(headlines)` for the guard check.

Expected Output:

```
Total headlines: 5 | Showing: 3
1. New Ai Law Passed
2. Budget Cuts Announced
3. School Reform Bill
```

6. Restaurant Rating Summary

Problem Statement:

- Given ratings = [3.8, 4.5, 2.9, 4.8, 4.1].
- Print the highest and lowest rating.
- Print ratings sorted highest to lowest (keep original unchanged).
- If the top rating is ≥ 4.5 , print 'Top restaurant qualifies for Featured badge!'; otherwise print 'No featured badge this week'.

Acceptance Criteria:

- Use `max()` and `min()` for highest and lowest.
- Use `sorted(ratings, reverse=True)` — do not modify original list.
- Store the top rating in a variable.
- Use `if/else` comparing top rating to 4.5.

Expected Output:

```
Highest: 4.8 | Lowest: 2.9
Ranked: [4.8, 4.5, 4.1, 3.8, 2.9]
Top restaurant qualifies for Featured badge!
```

7. To-Do App — Task Manager

Problem Statement:

- Start with tasks = ['Buy groceries', 'Call doctor', 'Pay bills'].
- Insert an urgent task 'Submit report' at the top of the list.
- Remove 'Call doctor' as it is completed.
- Print all remaining tasks converted to uppercase.
- If list has more than 2 tasks, print 'Busy day ahead!'; otherwise print 'Light day!'.

Acceptance Criteria:

- Use `insert(0, ...)` to add to the top.
- Use `remove()` to delete by value.
- Access each task by index and print with `.upper()`.

- Use `len()` in `if/else` for the day summary.

Expected Output:

```
SUBMIT REPORT  
BUY GROCERIES  
PAY BILLS  
Busy day ahead!
```

8. Sports Leaderboard Merger

Problem Statement:

- Given `region_a = [88, 74, 95, 61]` and `region_b = [79, 91, 85, 70]`.
- Merge both lists into one using `extend()`.
- Sort the merged list in descending order.
- Slice the top 3 scores.
- If the highest score is above 90, print 'A score above 90 made it to the podium!'; otherwise print 'No score above 90 this season'.

Acceptance Criteria:

- Use `extend()` to merge — do not use the `+` operator.
- Use `.sort(reverse=True)` for descending sort.
- Use slice `[:3]` for top 3 scores.
- Use `if/else` comparing `merged[0] > 90`.

Expected Output:

```
All scores: [95, 91, 88, 85, 79, 74, 70, 61]  
Top 3 finalists: [95, 91, 88]  
A score above 90 made it to the podium!
```

9. School Exam Score Report

Problem Statement:

- Given `scores = [[85, 90, 78], [92, 88, 95], [70, 75, 80]]`.
- Each inner list is one student's scores: Maths, Science, English.
- Print the Maths score (index 0) for all 3 students.
- Find and print the highest single score across the entire grid.
- Print the Science score (index 1) for Student 2 (index 1).

Acceptance Criteria:

- Access individual scores using double indexing: `scores[row][col]`.
- Print Maths scores for rows 0, 1, 2.
- Build a flat list using `extend()` or concatenation, then use `max()`.
- Print the correct Science score for `scores[1][1]`.

Expected Output:

```
Maths scores -> Student 1: 85 | Student 2: 92 | Student 3: 70
```

```
Highest score in class: 95
Student 2's Science score: 88
```

10. Inventory Stock Checker

Problem Statement:

- Given a list of product quantities: [0, 5, 12, 0, 3].
- Print the total number of products.
- Print the maximum and minimum stock quantity.
- Replace the first 0-quantity item by updating index 0 with value 8.
- Print the updated list.

Acceptance Criteria:

- Use `len()` for total count.
- Use `max()` and `min()` for stock extremes.
- Use direct index assignment: `stock[0] = 8`.
- Print the updated list.

Expected Output:

```
Total products: 5
Max stock: 12 | Min stock: 0
Updated stock: [8, 5, 12, 0, 3]
```
